BITS NLP ASSIGNMENT 2 | GROUP 112

English to Hindi Neural Machine Translation

Implementation, Evaluation and Application Report

Project type	End-to-end Neural Machine Translation system
Language pair	English (EN) to Hindi (HI)
Model	BiLSTM Encoder + Bahdanau Attention LSTM Decoder
Application	Streamlit web application
Submitted by	Group 112

Abstract

This project implements an English-to-Hindi neural machine translation system for digital-content localization and educational accessibility. The solution uses a sequence-to-sequence architecture with a bidirectional LSTM encoder, Bahdanau additive attention and an LSTM decoder. A Streamlit interface supports single-sentence translation, greedy or beam-search decoding, and batch translation from TXT or CSV input.

The system was trained on a cleaned subset of the AI4Bharat Samanantar English-Hindi parallel corpus. After normalization, length filtering and deduplication, 44,917 sentence pairs were retained and divided into training, validation and test partitions. On a 500-pair evaluation sample, the checkpoint achieved BLEU 17.09, chrF 22.66, ROUGE-L 35.17 and METEOR 20.77.

> **Scope note.** The checkpoint demonstrates a complete working pipeline and application, but translation quality is limited by the dataset size, vocabulary cutoff and frequent unknown tokens. Hindi examples in this report use valid Unicode text; square-box or otherwise invalid display artifacts are omitted.

Group Contribution

Student registration number	Name	Percentage of contribution
2025aa05883	KAREKE BUVIKA	100%
2025aa05337	BOLLIBATHULA SAHANI	100%
2025aa05567	SONAM	100%
2025aa05830	SARDESAI SHRISHTI SANJAY	100%
2025aa05906	ANSHUL SAHU	100%

1. Introduction and Objectives

English-authored documentation, government notices, educational material and digital content can be difficult to access for users who prefer Hindi. The project addresses this problem with a one-way EN-to-HI translation tool that exposes the trained model through a simple browser interface.

Objectives

Objective	Implementation outcome
Build an NMT model for English to Hindi.	Encoder-decoder PyTorch model with bidirectional encoding and additive attention.
Prepare reproducible training data.	Cleaning, filtering, splitting, vocabulary construction and fixed-length ID encoding.
Provide an accessible interface.	Streamlit app with single-sentence and batch upload workflows.
Measure translation quality.	BLEU, chrF, ROUGE-L, METEOR and targeted failure analysis.

Functional scope

The application accepts an English sentence directly or reads one sentence per line from a TXT file. It can also read a CSV file with an `english` column. Results are displayed alongside the source text and can be downloaded as a CSV file. Decoding can be switched between greedy search and beam search with a configurable beam width from 2 to 10.

2. Dataset and Preprocessing

Dataset

Property	Value
Source	AI4Bharat Samanantar corpus
Raw pull	Approximately 60,000 parallel English-Hindi pairs
Cleaned pairs	44,917
Split	35,933 train / 4,492 validation / 4,492 test
Sentence length	5 to 30 tokens per language before special tokens
Vocabulary cutoff	`min_freq=4` , built only from training data

Preprocessing sequence

The pipeline in `src/preprocess.py` and `run_pipeline.py` applies NFC Unicode normalization, URL and HTML-entity removal, zero-width formatting-character removal from Hindi, whitespace collapsing, English-only lowercasing, length filtering and pair deduplication. Each sequence is wrapped with `<sos>` and `<eos>` , encoded with the training vocabulary and right-padded to a fixed length of 34 tokens.

Data flow

Raw corpus → cleaned sentence pairs → 80/10/10 split → training-only vocabularies → integer ID matrices → PyTorch DataLoaders → trained checkpoint → CLI and Streamlit inference.

3. Improved Project Structure

The repository is organized around the lifecycle of the system: data preparation, model training, evaluation, inference and user-facing application. The following expanded view clarifies the responsibility of every major file and separates source data, generated artifacts and execution entry points.

```
nlp_assignment_2/
|-- README.md                       Project description, setup and usage
|-- requirements.txt                Python dependencies
|-- run_pipeline.py                 Download, clean, split and encode data
|-- train.py                        Model classes, training loop and decoding
|-- evaluate.py                     Metrics and failure analysis
|-- infer.py                        Command-line translation interface
|-- app.py                          Streamlit web application
|-- test_input.txt                  Sample batch input, one sentence per line
|-- data/                           Reproducible model-ready data artifacts
|   |-- config.json                 Sequence length and vocabulary metadata
|   |-- src_vocab.json              English token-to-ID vocabulary
|   |-- tgt_vocab.json              Hindi token-to-ID vocabulary
|   |-- train_ids.json              Encoded and padded training pairs
|   |-- val_ids.json                Encoded and padded validation pairs
|   `-- test_ids.json               Encoded and padded test pairs
|-- models/                         Trained model and experiment artifacts
|   |-- nmt_model.pt                Best saved PyTorch checkpoint
|   |-- training_log.json           Hyperparameters and per-epoch losses
|   |-- loss_curve.png              Training/validation loss visualization
|   `-- eval_report.json            Evaluation metrics and analysis
|-- src/
|   |-- preprocess.py               Cleaning, tokenization and encoding
|   |-- vocab.py                    Vocabulary implementation
|   `-- __init__.py                 Python package marker
`-- snaps_/                          Application and training evidence
    |-- inference .png               Command-line inference output
    |-- with_beam.png                Single-sentence input/output screen
    |-- batch_file.png               Batch upload and result table
    |-- training.png                 Training console evidence
    |-- loss_curve.png               Loss-curve evidence copy
    `-- bits_osha.png                Final application capture
```

Execution responsibilities

Stage	Entry point	Primary outputs
Data preparation	`python run_pipeline.py --min_freq 4`	Clean splits, vocabularies and encoded IDs in `data/`
Training	`python train.py`	Checkpoint, loss history and curve in `models/`
Evaluation	`python evaluate.py --n_samples 500`	Metrics and failure analysis in `models/eval_report.json`
Inference	`python infer.py`	Terminal translation or CSV output
Application	`streamlit run app.py`	Browser-based translation interface at localhost:8501

4. Model and Implementation Details

Encoder

The encoder embeds source tokens using a 128-dimensional embedding table and processes them with a single-layer bidirectional LSTM of hidden size 256. The final forward and backward hidden and cell states are concatenated and projected through tanh-activated linear layers to initialize the single-direction decoder.

Attention and decoder

At each decoding step, Bahdanau additive attention scores every encoder output against the current decoder hidden state. The resulting context vector is concatenated with the current target embedding and passed to an LSTM. The decoder output, attention context and embedding are combined by a linear projection into the Hindi vocabulary distribution.

Training configuration

Setting	Value
Embedding / hidden size	128 / 256
Batch size	128
Learning rate	0.0005, resumed run
Dropout	0.4
Loss	Token cross-entropy with label smoothing 0.1; padding ignored
Regularization	Weight decay 1e-4 and gradient clipping at 1.0
Teacher forcing	0.6
Scheduler	ReduceLROnPlateau, factor 0.5, patience 2
Early stopping	Patience 6; best validation loss 5.8715 at epoch 20

Decoding safeguards

The application supports greedy decoding and length-normalized beam search. During decoding, an immediate repeated-token block and no-repeat-trigram constraint reduce repetition loops. The evaluation report records a 0.0 percent repetition-loop rate for the 500-sample evaluation.

5. Training and Evaluation

```
(venv) cloud@2025aa05830:~/Desktop/NLP2/nlp_assignment_2-main$ python train.py
Device: cpu | Src vocab: 12370 | Tgt vocab: 11354
Train batches: 305 | Val batches: 39
Epoch 01/15 | Train Loss: 6.7681 | Val Loss: 6.6136 | LR: 1.00e-03 | Time: 287.7s
  -> Saved new best checkpoint (val_loss=6.6136)

Epoch 02/15 | Train Loss: 6.3309 | Val Loss: 6.4294 | LR: 1.00e-03 | Time: 289.8s
  -> Saved new best checkpoint (val_loss=6.4294)
Epoch 03/15 | Train Loss: 6.1319 | Val Loss: 6.3422 | LR: 1.00e-03 | Time: 288.9s
  -> Saved new best checkpoint (val_loss=6.3422)
Epoch 04/15 | Train Loss: 6.0152 | Val Loss: 6.2920 | LR: 1.00e-03 | Time: 288.1s
  -> Saved new best checkpoint (val_loss=6.2920)
Epoch 05/15 | Train Loss: 5.9206 | Val Loss: 6.2476 | LR: 1.00e-03 | Time: 288.7s
  -> Saved new best checkpoint (val_loss=6.2476)
Epoch 06/15 | Train Loss: 5.8434 | Val Loss: 6.2084 | LR: 1.00e-03 | Time: 288.4s
  -> Saved new best checkpoint (val_loss=6.2084)
Epoch 07/15 | Train Loss: 5.7740 | Val Loss: 6.2001 | LR: 1.00e-03 | Time: 287.8s
  -> Saved new best checkpoint (val_loss=6.2001)
Epoch 08/15 | Train Loss: 5.7056 | Val Loss: 6.1589 | LR: 1.00e-03 | Time: 286.5s
  -> Saved new best checkpoint (val_loss=6.1589)
Epoch 09/15 | Train Loss: 5.6455 | Val Loss: 6.1648 | LR: 1.00e-03 | Time: 287.4s
  -> No improvement for 1/4 epoch(s)
Epoch 10/15 | Train Loss: 5.5858 | Val Loss: 6.1037 | LR: 1.00e-03 | Time: 287.6s
  -> Saved new best checkpoint (val_loss=6.1037)
Epoch 11/15 | Train Loss: 5.5172 | Val Loss: 6.1002 | LR: 1.00e-03 | Time: 288.0s
  -> Saved new best checkpoint (val_loss=6.1002)
```

Figure 1. Training console evidence showing epoch-wise loss and checkpoint selection.

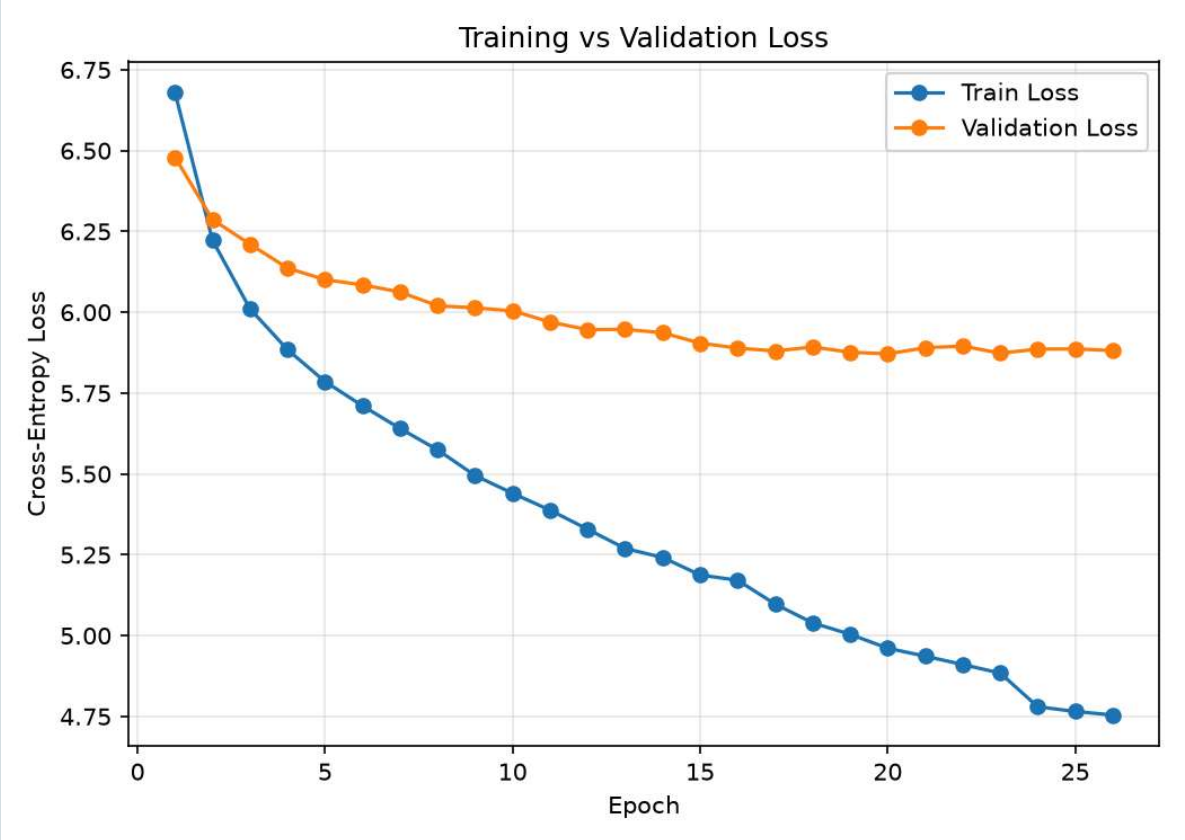

Figure 2. Training and validation loss curve from the submitted checkpoint.

Evaluation results

Metric	Score	Interpretation
BLEU	17.09	Moderate n-gram overlap on the scored sample.
chrF	22.66	Character-level similarity, useful for morphologically rich Hindi.
ROUGE-L	35.17	Longest-subsequence similarity between reference and output.
METEOR	20.77	Alignment-based translation similarity.

Failure analysis	Result
Short sentences, under 10 tokens	157 items; average chrF 22.81
Medium sentences, 10-20 tokens	268 items; average chrF 23.11
Long sentences, over 20 tokens	75 items; average chrF 22.69
Outputs containing unknown tokens	88.2 percent
Repetition-loop rate	0.0 percent

6. Working Application Evidence

6.1 Input screen and sample submitted

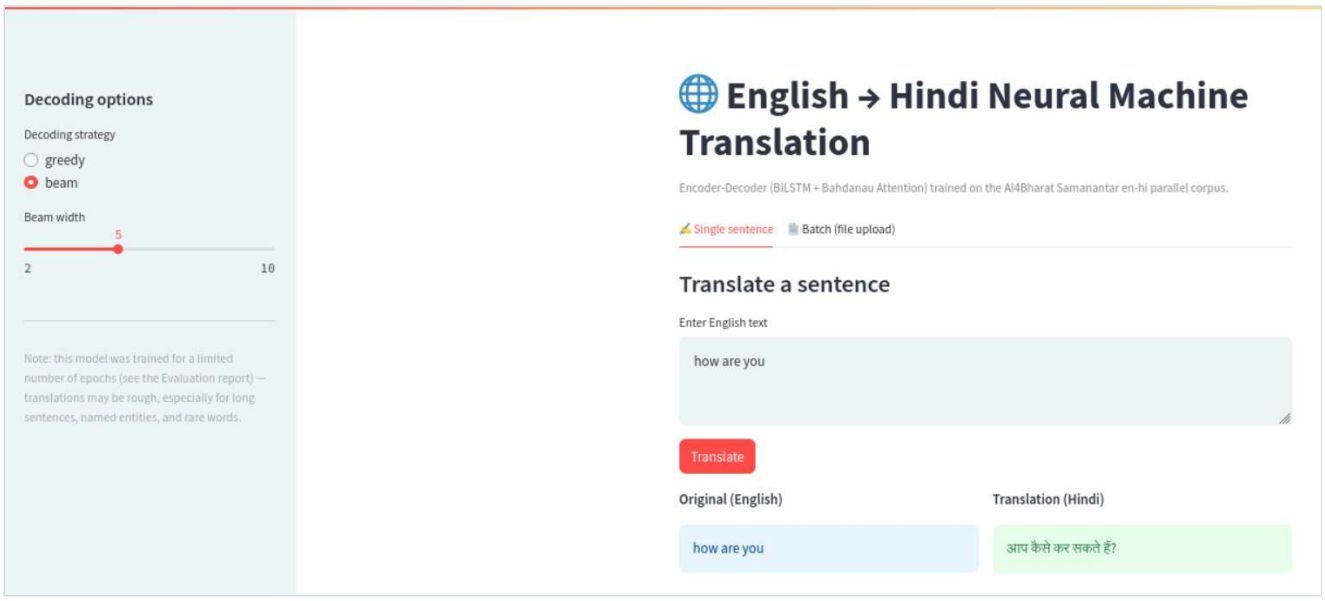

Figure 3. Single-sentence input screen with the sample “how are you”, beam decoding selected, and the translation result visible.

The screen demonstrates the main interaction: the user enters English text in the text area, selects a decoding strategy in the sidebar and submits the request using the Translate button. The original English sentence and generated Hindi text are rendered in separate result panels.

6.2 Batch sample submission and generated output

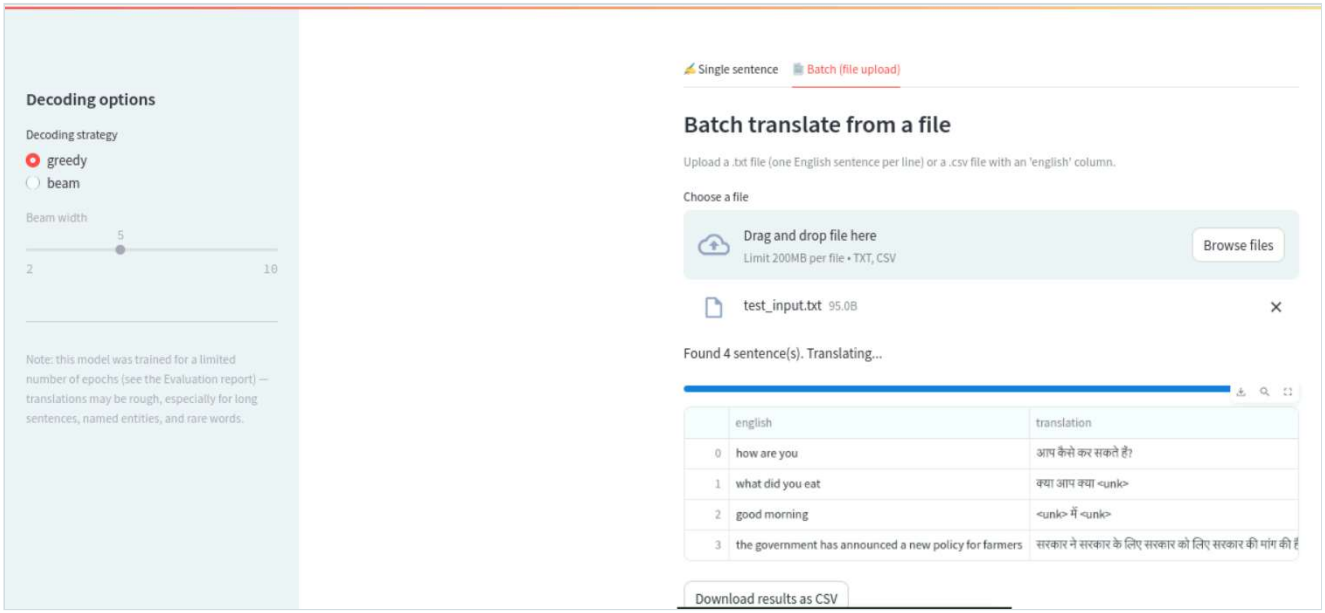

Figure 4. Batch workflow showing `test_input.txt` uploaded and a generated English-to-Hindi result table.

The batch path reads four non-empty lines from the uploaded text file, translates each line and presents the results in a table. The same table can be downloaded as a UTF-8 CSV file for downstream use.

Inference Output

test_output.txt

```
EN : how are you
OUT: कैसे आप क्या कर रहे हैं?

EN : what did you eat
OUT: क्या क्या <unk> क्या क्या है?

EN : good morning
OUT: <unk> के लिए <unk>

EN : the government has announced a new policy for farmers
OUT: सरकार सरकार ने किसानों सरकार सरकार की सरकार की है।
```

Figure 5. Inference output showing the submitted English examples and generated Hindi translations.

7. Observations and Limitations

Observations

1. The full workflow is operational: data artifacts and the trained checkpoint are available, the model loads in the application, and both single and batch interfaces are implemented.

2. The loss curve shows a strong early reduction in training loss, while validation loss improves more slowly and reaches its best value near epoch 20. Later epochs produce no meaningful validation improvement, which justifies early stopping.

3. Short and medium inputs have similar chrF performance, while long inputs are slightly weaker. This is consistent with the sequential model needing to preserve more context over longer and more complex sentences.

4. The no-repeat constraints are effective against decoding loops, but they cannot solve vocabulary coverage. Unknown-token output remains the primary quality limitation.

Limitations

The model was trained on approximately 36,000 training pairs after cleaning, with a minimum vocabulary frequency of four. Rare words and named entities can therefore map to unknown tokens or be mistranslated. The submitted checkpoint is a compact assignment-scale model rather than a production translation system. The application also relies on the included local artifacts; regenerating the dataset requires internet access to Hugging Face.

8. Conclusion

An end-to-end English-to-Hindi NMT system was implemented, evaluated and integrated into a usable Streamlit application. The repository now clearly separates preparation, training, evaluation, inference, application code and generated artifacts. The submitted model produces valid Hindi output for some common inputs and supports useful operational workflows such as beam decoding and batch CSV export.

The measured scores and failure analysis show that the project meets the implementation objective while also making its quality boundaries explicit. The strongest next step is subword-based modeling with broader data, which should directly address the observed 88.2 percent unknown-token rate and improve named-entity and long-sentence translation.

9. References

[1] AI4Bharat, "Samanantar: English-Hindi parallel corpus," Hugging Face Datasets, https://huggingface.co/datasets/ai4bharat/samanantar

[2] Bahdanau, D., Cho, K. and Bengio, Y., "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR, 2015.

[3] PyTorch Documentation, "Sequence Models and Recurrent Neural Networks," https://pytorch.org/docs/

[4] Streamlit Documentation, "Build and deploy data apps," https://docs.streamlit.io/

[5] Project README and implementation files: `run_pipeline.py`, `train.py`, `evaluate.py`, `infer.py` and `app.py`.

Appendix: Final Application Capture

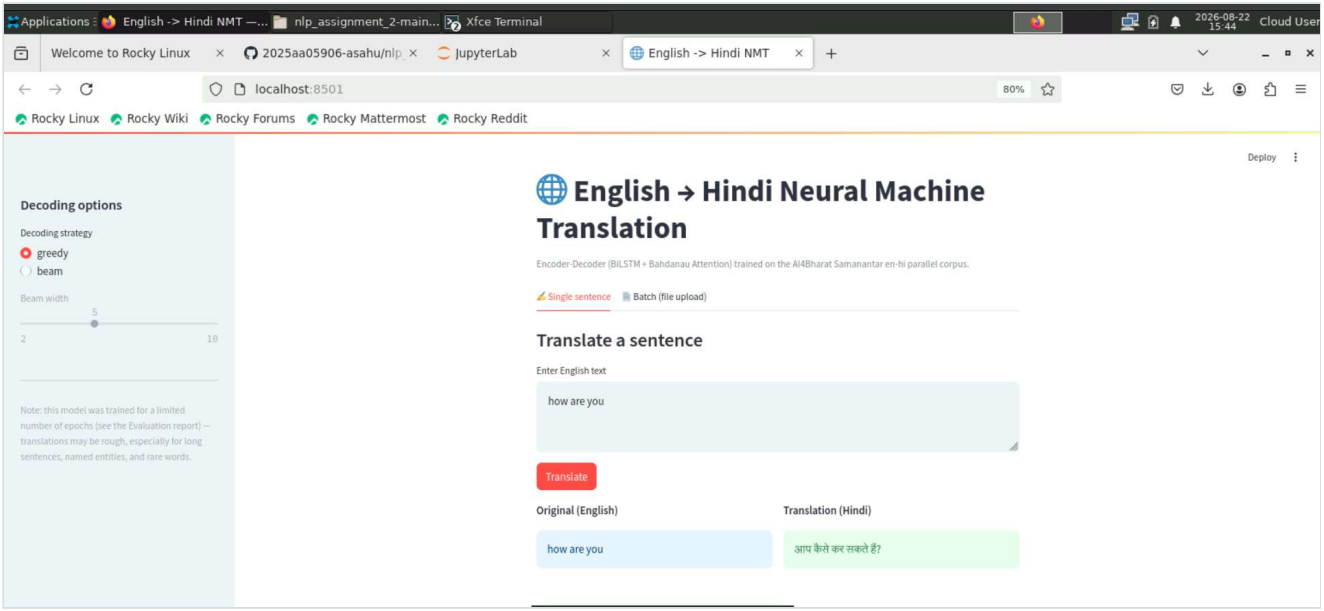

Figure 6. Final application capture: Streamlit English-to-Hindi translation interface with a submitted example and generated output.